firm Cohere, points out, a big difference between JavaScript libraries and Ruby ones is that the JavaScript libraries often consider boilerplate to be a good thing, whereas Ruby libraries try to avoid it.)

So when using a thunk, the call might still look like `dispatch(doSomething())` but would differ by returning a function as in:

```
const doSomething = () => {
      return async (dispatch) => {
        await doSomethingAsynchronous()
        dispatch({ type: "doSomethingElse" })
  }
}
```

Adding Redux Thunk into the mix changes Redux slightly. Once Redux Thunk is added, when Redux receives an argument to a dispatch call that is a function rather than an action type, Redux executes the function passed to it to get the return value, and passes that returned function a `dispatch` argument (and, as we'll see, a `getState` argument). By doing this, Redux Thunk controls the execution of our action and allows us to do asynchronous stuff and coordinate it with our reducer while still using the same `dispatch` method to communicate with it—the two steps we were doing before, combined.

For our purposes, what we need to do is add a mechanism to coordinate our ActionCable subscription with our state, and then convert our existing actions that deal with the subscription into Redux Thunk functions.

Redux Thunk is already included as part of the Redux Toolkit, and it's already mixed in to our store from our `configureStore` call; that all happens behind the scenes. The reason why the `dispatch` type is dynamic for our `AppDispatch` is because the type depends on what Redux middleware add-ons, like Redux Thunk, might be added. Redux Thunk is added by default, but there's an ecosystem of add-ons we might include in the future.

We have some imports to add:

chapter_12/02/app/javascript/contexts/venue_context.ts